CYVL HACKATHON · JUNE 13, 2026 · EXECUTION RUNBOOK

Sewershed: step by step.

Every step from doors-open to submission, in order.

> **Read once tonight. Follow tomorrow. Don't improvise the plan; improvise inside it.**

ALL whole team · MAP / DATA builder roles · ML CV track · PITCH demo & story

BUILD WINDOW
9:30 AM – 5:30 PM

SUBMISSION
GitHub repo by 5:30 PM

JUDGING
Booth 5:30 · Top-5 stage 6:15

Phase 0 — Tonight (before sleep)

1. **0.1** Confirm every teammate accepted the Cyvl platform email (check junk for the verification code). ALL
2. **0.2** Everyone logs into cyvl.app once and pans around Somerville for 10 minutes. ALL
3. **0.3** Create a free account at `viewer.autodesk.com` and upload any test file. Never first-login on stage. PITCH
4. **0.4** Verify the data folder is intact: 11 GeoJSON files in `cyvl_hackathon/data/` (~30 MB). Zip it and post in the team chat so everyone has a copy. DATA
5. **0.5** Charge laptops. Pack: chargers, power brick, USB-C Ethernet adapter if you have one, water.
6. **0.6** Sleep. Nothing on this list is worth less sleep.

Phase 1 — Kickoff (9:00–9:30)

1. **1.1** Collect the team API key ($200 Anthropic credits) and Cyvl key at check-in. Put both in the team password vault / pinned chat message. Never commit them. ALL
2. **1.2** Confirm team + roles in 5 minutes: Map/UI · Data/Join · ML · Pitch. One person may hold two hats; nobody holds three. ALL
3. **1.3** Create the GitHub repo (`sewershed`), private until submission rules say otherwise. DATA
4. **1.4** Scaffold: `npx create-next-app@latest sewershed --typescript --tailwind --app` , push, link Vercel, confirm the hello-world URL loads on a phone. MAP
5. **1.5** Copy the `data/` folder into the repo under `/public/data/` . DATA
6. **1.6** Connect Claude Code to the Cyvl MCP per their setup doc. Run one test query ("pavement condition near Highland Ave"). DATA
7. **1.7** Vote: demo catchment = Mystic side / Somerville Marginal (S2) — the dry-run showed Alewife has only ~5 combined pipes (unusable on stage) while S2 has ~667 AND carries the 95 committed acres and the 2027 Mystic outfall construction. ALL

> **Done when:** public URL live · MCP answers a query · roles assigned · keys stored.

Phase 2 — First light (9:30–10:30)

Track A: the red/green map MAP

1. **2.1** Add Leaflet (`preferCanvas: true` — 6,400 polylines need canvas, not SVG) + OSM tiles. Center on Somerville (42.3876, -71.0995), zoom 14.
2. **2.2** Load `sanitary_mains.geojson` ; style `WATERTYPE === 'Combined'` red (weight 3), everything else gray.
3. **2.3** Load `storm_mains.geojson` green. Load `sewer_subsystems.geojson` as dashed boundary polygons.
4. **2.4** Add the "2,438 combined segments" counter (compute it live from the loaded file, don't hardcode).
5. **2.5** Deploy. This screen alone is already a demo.

Track B: Cyvl API smoke test DATA

1. **2.6** Pull pavement scores for the Alewife bbox: `GET /pavement/scores?project_id=…&bbox=…` . Save raw JSON to `/public/data/cyvl/` .
2. **2.7** Pull distresses filtered to utility-cut patching. Pull assets (catch basins, manholes, ramps) for the same bbox.
3. **2.8** Note the exact field names Cyvl returns (don't assume; read one feature).
4. **2.9** Re-derive the two headline stats via API (utility-cut patching % and PCI average) so every number on stage is API-traceable, not screenshot-traceable. DATA

> **Done when:** red/green map is deployed · real Cyvl JSON files are on disk with known schemas.

Phase 3 — The join & the score (10:30–12:30)

1. **3.1** Write `scripts/join.ts` as constrained nearest-neighbor map matching: build a `flatbush` R-tree over street centerlines once → k=5 nearest candidates per combined pipe → filter (dist < 25 m, same catchment) → score (distance + street-name similarity) → assign or orphan. DATA
2. **3.2** Same matcher for Cyvl observations → streets (15 m threshold; pavement records carry `address_st` for the name term). Attach: avg pavement score, patch count (joined by `inspect_id`, not geometry — verified), utility asset density, sidewalk `Last_SCI`, ramp gaps. (Depth fields unpopulated — verified.) DATA
3. **3.3** Write `lib/score.ts` — the pure function (weights 30/25/20/15/10). Unit-test with 3 hand-made segments; add the ±10% weight-sensitivity check (top-10 stability number for Q&A). DATA
4. **3.4** Run the script → emit `public/data/segments_scored.json`. Sanity-check the top 10 by eye on the map: do they look plausible? Old streets, bad pavement? DATA
5. **3.5** In parallel: build the catchment sidebar (7 progress bars computed from pipe data) and the ranked-list panel against a mocked `segments_scored.json` with the same shape. MAP
6. **3.6** Swap mock for real file. Click a catchment → list filters. MAP

If the matcher fights back past 11:45: drop to nearest-only within 25 m (skip constraints), or name-match as last resort; note it honestly, move on. The ranking needs plausible attribution, not survey-grade.

> **Done when:** `segments_scored.json` exists from real data · map colors streets by score · ranked list renders.

Phase 4 — Evidence & the CV track (12:30–14:00)

Track A: detail card MAP

1. **4.1** Click street → card: readiness score, factor breakdown bars, pipe install year, cut depth, sidewalk SCI.
2. **4.2** Fetch the street's Cyvl photo via `GET /assets/{id}/imagery`; cache URLs into the scored file.
3. **4.3** Bundling flags as chips: "+2 ADA ramps", "+sidewalk rebuild", "6 utility cuts on block".
4. **4.4** `/api/explain`: Claude streams a 2-sentence justification from the raw factors. Cache responses per segment.

Track B: distillation pipeline ML

1. **4.5** Gather crops via per-asset `image_url` on all 381 Somerville catch basins (verified attached to every record); optionally enrich rare classes via embeddings search on the **Boston Full** project (works there: 236,983 images; Somerville has 0).
2. **4.6** Claude vision auto-labels all crops (clear/clogged/damaged/uncertain) with the class definitions in the prompt.
3. **4.7** Team audits 20 labels (5 min). Record the agreement number — it goes on a slide.
4. **4.8** Train the student: CLIP/DINOv2 features + small head (fast path) or fine-tune ViT on GPU (NVIDIA path). Hold out 20%; abstain below confidence (renders "unverified" on the map).
5. **4.9** Batch-score all 381 catch basins → new map layer toggle. Save accuracy + 2 honest misclassification examples for the demo.

> **Done when:** click any top-10 street and see photo + factors + AI explanation · catch-basin condition layer exists with a quoted accuracy.

Phase 5 — The wow (14:00–15:30)

1. **5.1** Time-lapse: client-side loop over the ranked list. Each tick = flip next street green, advance year counter (2026→2056 mapped over the list), increment the savings counter (use a flat per-segment dig-once estimate; cite the "up to a third" coordination figure as the basis). Play/pause button. MAP
2. **5.2** Budget slider: greedy take from ranked list until budget spent; recolor selected streets; update totals. Client-side only. MAP
3. **5.3** Weights panel: five sliders, re-rank live (the score function runs in the browser on the precomputed factors). DATA
4. **5.4** 311 validation layer: pull Somerville 311 sewer/flood complaints (SODA API), render as dots, toggle in the layer list. DATA
5. **5.5** Wire the Anthropic explainer into the detail card if not done in 4.4. DATA

> **Done when:** press play and the city heals itself in ten seconds while dollars count up.

Phase 6 — Exports & polish (15:30–16:30)

1. **6.1** Export: ranked dig order as CSV and GeoJSON downloads. DATA
2. **6.2** DXF: `dxf-writer` emits the top-N street polylines with rank labels. Test-open the file in Autodesk Viewer NOW, not on stage. DATA
3. **6.3** Polish pass: typography, layer legend, mobile check on a phone, empty-state handling. MAP
4. **6.4** Screen-record the two live-network moments (image search, DXF open) as backup videos onto the pitch laptop. PITCH
5. **6.5** README: problem, data sources (all real, listed), how to run, screenshots. Judges read READMEs. PITCH

> **Done when:** a stranger with the URL can use the tool unaided · DXF verified in Viewer · backups recorded.

Phase 7 — Submission & pitch (16:30–17:30)

1. **7.1** Freeze features at 16:30. Bugs only after this line. ALL
2. **7.2** Submit the GitHub repo per their instructions. Verify it's accessible from an incognito window. DATA
3. **7.3** Rehearse the booth loop 3 times, timed (target 2:00): red city → time-lapse → evidence card → live search → DXF close. Driver + narrator assigned. ALL
4. **7.4** Prepare the 4-minute stage version: add Spring Hill story, the scope-honesty beat, scales-to-Boston close. PITCH
5. **7.5** Write the demo URL big on a card at the booth. Judges open it on their phones. PITCH

> **Done when:** repo submitted · loop runs at 2:00 flat · URL card on the table.

Cut order and emergencies

If behind at…	Cut
14:00	CV track (downgrade NVIDIA angle to embeddings-search-only, still live)
15:00	Weights panel, then 311 layer
15:45	DXF export (keep CSV; mention Civil 3D path verbally)
16:15	Claude explainer (factor bars carry the card alone)
Never cut	Red/green map · ranked list · detail card with photo · time-lapse

Emergency	Response
Cyvl API down	Flag staff (offline dataset promised); meanwhile everything precomputed keeps working
Live image-search demo beat	Run it against project "Boston Full" (8d8f8cd6…, embeddings verified) — not the Somerville project (0 embeddings)
Venue wifi dies	App is static + local — run from localhost; backups cover the two live moments
Join produces garbage	Fall back to per-catchment aggregates (coarser but honest); the story survives
Laptop dies at booth	Public URL on any phone + backup videos on second laptop

> **One rule above all:** something deployable exists at every hour boundary. Never be 30 minutes from a working demo.